Irreversible Metadata Scrubber — Client Progress Report

Status: Phases 0, 1 and 2 complete · Phase 3 open **Scope of this report:** what has been built, what has been tested and proven, and what comes next

---

1. Executive summary

Every file you send someone carries hidden information you did not intend to send: where a photo was taken, which phone or program made it, when it was written, who edited it, and often a small preview image or an earlier draft buried in the bytes. This project builds a tool that **removes that hidden information so it cannot be recovered again** — not merely deleted from view, but gone from the file even when someone examines it byte-by-byte with forensic software.

Where we are today. The tool works, ships as a command-line program, and handles five file formats end to end: **JPEG, PNG, MP3, FLAC and M4A**. It is backed by **243 automated tests** that run on every code change across four versions of Python, and by a measurement system that re-proves every published claim from scratch on each run — if a result stops being true, the build fails.

What makes this different from existing tools. Mature tools already delete tags well; we are at parity there and say so. Our contribution is three things nobody else offers together:

1. **You choose the trade-off.** Three cleaning levels, from "don't touch the picture at all" to "make the file untraceable", so the user picks the strength that matches the risk they actually face.
2. **We defeat the encoder fingerprint** — the hidden signature that reveals *which program or device made a file* even after every tag is gone. For two formats (PNG and FLAC) we now do this at **zero quality cost**: identical pixels, identical audio, down to the last bit.
3. **Every claim is measured, and every limit is published.** We maintain a written register of what the tool cannot do, in plain language, and the automated build publishes it alongside the successes. There is no claim in this project that is not backed by an experiment.

What is next. Phase 3 — **documents (PDF, then Word/OOXML)**. This is the hardest and most valuable phase: it targets the single most famous document-leak in existence (a PDF's deleted earlier drafts remaining inside the published file) and a known failure that every surveyed competing tool shares (Word revision IDs that survive every scrubber on the market). Groundwork is already measured and the design decision that governs the phase has been taken.

---

2. What "irreversible" means here, and who we defend against

The word does a lot of work, so we pin it down precisely.

Irreversible = forensically unrecoverable from the cleaned file itself. Not hidden, not blanked, not overwritten with spaces. If the original file still sits in the user's own folder afterwards, that is a workflow question for the user — not a property of the file we produce.

We defend against a **medium-tier adversary**: a journalist, a competitor, or an amateur investigator using off-the-shelf forensic tools. Not a nation-state intelligence agency. Being explicit about this is what lets us make claims we can actually prove.

That adversary comes in two strengths, and beating the first is easy while beating the second is the real work:

	Who they are	What they read
A1 — the tag snoop	Opens the file and reads what is written in it	GPS coordinates, camera make and model, author name, timestamps, editing software, embedded thumbnails, comments
A2 — the fingerprint snoop	Assumes every tag is already gone, and studies *how* the file was built	The compression settings, the ordering of internal structures, the encoder's habits — enough to say "this came from an iPhone" or "this came from Photoshop" with no tag present at all

A2 is the interesting adversary. It is like identifying a typewriter from the shape of its letters when there is no name on the page. Most privacy tools have no answer to it.

A third level, A3, is where the investigator already holds the *original* file and compares. We name it for completeness; it is outside what any scrubber can defeat.

---

3. The three cleaning levels

The user chooses how much fidelity to spend on anonymity. This choice is the product.

Level	What it does	What it costs
F1 — bit-preserving	Removes every piece of metadata; the picture or audio is left **completely untouched**, byte for byte.	Nothing.
F2 — lossless rebuild	Removes metadata **and** rebuilds the file's packaging through one standard route, so the packaging no longer reveals its origin. Content still perfect.	Nothing — genuinely zero.
F3 — full re-encode	Re-records the picture or sound through one standard encoder, so every file we produce carries the same signature as every other.	A small, generally invisible quality loss.

The tool identifies file types **by their actual contents, not their filename**, and is **fail-closed**: when it cannot fully guarantee a result, it refuses and writes nothing rather than hand back a file that merely looks clean.

```
python -m src.scrub --fidelity F1|F2|F3 <input> <output>
```

---

4. What has been built

4.1 The foundation (Phase 0)

Before any single file format, we built the shared machinery — this is why later formats land quickly and consistently rather than each being a bespoke effort:

- **A differential-testing harness.** The core verification idea: take two files with identical content but different metadata, clean both, and compare. Anything that still differs is a leak, by definition. This is the pass/fail gate for every format; nothing ships on visual inspection.
- **A test corpus generator** that injects metadata into *every known hiding place* at once, not just the obvious tags — so passing means all duplicates were cleared.
- **Shared standards modules**, written once and called by every format: EXIF/TIFF, XMP, ICC colour profiles, IPTC, ISO base media (the box format behind M4A, MP4 and HEIC), and a perceptual quality gate. Copy-pasting these per format is how tools develop leaks; we deliberately do not.
- **A scrubber-fingerprint guard** — a check that *our own tool* leaves no signature: no "cleaned by" string, no distinctive padding, no timestamps, deterministic output. It has already caught us twice and forced changes.
- **Magic-number dispatch**, so every format plugs into one entry point.

4.2 Images — Phase 1, complete

- **JPEG** at all three levels, including full removal of the embedded thumbnail (a small copy of the photo, with its own separate GPS and camera data — the most commonly missed hiding place in the industry).
- **PNG** at F1 and F2. PNG needs no F3: it is lossless by nature, so **F2 already achieves untraceability at zero cost**.

4.3 Audio — Phase 2, complete

- **MP3** — tag removal at F1, and a canonical re-encode at F3 that erases the encoder fingerprint. Proven against a genuinely *different* encoder, not just a different program driving the same one.
- **FLAC** — F1 and F2, with **untraceability achieved losslessly**: the audio comes out bit-identical, mathematically verified through the format's own internal checksum.
- **M4A** — F1, F2 and F3, including surgery on the file's internal box structure with sample-table repair. **This is the format MAT2, the leading open-source privacy tool, refuses to process at all.** A user who hands MAT2 an `.m4a` gets nothing back; ours handles it at all three levels.

4.4 The engineering around it

- **13,800 lines** of Python across the tool, harness, experiments and reporting.
- **Continuous integration** on every change: code quality, the full test suite on Python 3.11 / 3.12 / 3.13 / 3.14, a coverage gate, and — most importantly — a job called *"Published results still true"* that re-measures every published claim from scratch and **fails the build if a result has drifted**. Honesty is enforced by machine, not by memory.
- **A plain-language QA report** published automatically on every run, written for a non-technical reader: a verdict banner, a pipeline diagram coloured by what actually happened, before-and-after evidence on real sample files, and a "what we can't do yet" section rendered directly from our limits register.

---

5. What has been tested, and what it proved

5.1 Results (measured, never assumed)

Format	A1 — tags removed (F1/F2/F3)	A2 — fingerprint erased (F1/F2/F3)
JPEG	pass / pass / pass	fail / fail / **pass**
PNG	pass / pass / n-a	fail / **pass** / n-a
MP3	pass / n-a / pass	fail / n-a / **pass**
FLAC	pass / pass / n-a	fail / **pass** / n-a
M4A	pass / pass / pass	fail / narrowed to one channel

Reading this table: the tag snoop (A1) is defeated everywhere, at every level. The fingerprint snoop (A2) is defeated by rebuilding — and **twice now at no quality cost at all** (PNG and FLAC), while JPEG and MP3 must spend one re-encode to get there.

The dividing line is a real technical finding, not a limitation of effort: if the fingerprint lives in a *separable layer* (PNG's compression, FLAC's metadata blocks) it can be normalised for free. If it is *baked into the compressed content itself* (JPEG's quantization tables, MP3's encoder header and bitrate contour), removing it costs a generation of quality. That distinction is a genuine contribution and it is what the Pareto matrices exist to express.

5.2 The experiments behind the table

Each A2 claim comes from a named experiment with a stated method, not from reasoning:

Experiment	The question it answers	Result
E3	Does JPEG's compression signature identify the producing program?	Yes at F1/F2 — erased at F3
E-LAME	Do MP3 file headers still identify the encoder after cleaning?	All producers collapse to one signature
E-ENGINE	Harder: with headers normalised, can the encoder be recovered from **the sound itself**?	0.89 → 0.53 at 44.1 kHz, 0.94 → 0.58 at 22.05 kHz (chance = 0.50)
E-FLAC	Can FLAC be made untraceable **without** touching the audio?	Yes — bit-identical audio, verified by checksum
E-M4A-AUDIO	Does the original recording's quality setting survive in the sound?	0.88 on untouched files → chance (0.50) after cleaning

Two methodological commitments run through all of them, and both were adopted after they caught us:

- **Verdicts are statistical significance tests, not thresholds.** An earlier version used a fixed accuracy margin; with a small sample that turned random noise into a verdict, and the same experiment landed on opposite answers on two different machines. We enlarged every peer set until the answer stopped moving and now report p-values.

- **Every experiment asserts its own controls.** If an attack cannot identify the producer of an *untouched* file, its failure on a cleaned one proves nothing — that case is reported as **inconclusive**, never as a pass. We apply this against our own interest: the M4A engine-identity question is reported as *unanswerable with this corpus* rather than counted as a win.

5.3 How we compare to the tools that already exist

Measured directly against ExifTool, MAT2 and jpegtran on our own torture corpus:

Capability	Ours	ExifTool	MAT2	jpegtran
Removes all named metadata	✅	✅	✅	✅
Keeps the picture pixel-perfect	✅	✅	❌ always lossy	✅
No cumulative damage when run repeatedly	✅	✅	❌ degrades each pass	✅
Erases the encoder fingerprint (untraceable)	✅	❌	⚠️ only as a side effect	❌
Lossless *and* untraceable	✅	❌	❌	n/a
You choose the fidelity for the threat you face	✅	❌	❌	❌
A measured guarantee matrix	✅	❌	❌	❌
Handles M4A audio at all	✅	⚠️ tags only	❌ **refuses**	n/a
Clears a FLAC third-party data block	✅	⚠️ not by default	❌ **leaves it**	n/a

Two concrete findings worth naming. Running the leading tools on our corpus, **MAT2 leaves an arbitrary third-party data block inside FLAC files completely untouched** — a general-purpose container any application can write anything into, forwarded by a privacy tool that never inspected it. And **ExifTool, run on an M4A, leaves the file's creation and modification times behind**, because they are structural fields rather than tags.

Equally, the honest half: on plain tag removal from a normal JPEG, we are at **parity** with mature tools, not ahead. That is reported as parity.

5.4 The limits we publish

We maintain a register of twelve known limits in plain language, and the automated report renders it verbatim on every run — a limit recorded there reaches every reader automatically. The headline ones:

- **A cleaned file is recognisably cleaned.** Everything we produce comes out in the same standard shape, so an observer can tell a scrubber was used. They cannot tell *which* program, phone or camera made the original — and that second question is the one this tool promises to answer. Hiding that a file was cleaned is a different problem.
- **Camera sensor noise (PRNU) and microphone/room signatures cannot be removed** by any metadata tool that exists. They live in the picture and the sound themselves. We document these as structural impossibilities rather than pretending they are solved.
- **A more sophisticated investigator than the one we built might still succeed.** Our audio attack studies the frequency profile; published research also studies the encoder's internal arithmetic, which we have **not** tested against. We name the gap rather than implying it away.
- **Some unusual audio files we refuse outright** rather than half-clean them. Nothing leaks, but the user gets no output — broader support is planned.

---

6. Near future — Phase 3: documents (PDF, then Word)

This phase is open and is the most valuable one in the plan. Documents are where the real-world disclosures happen.

6.1 The two named targets

PDF incremental-update history. A PDF is edited by *appending* — the old version stays in the file underneath the new one. Text "deleted" three revisions ago is still there in full. This is the mechanism behind the famous redaction failures where published documents were read straight through the black boxes. Our answer is that a clean must **rewrite** the document to a single revision, never append, and then prove by carving the output that no earlier revision survived.

Word revision-save IDs (RSIDs). Word stamps identifiers throughout a document that link individual edits to editing sessions. Published research shows they **survive every surveyed scrubber, MAT2 included**. Clearing them is a capability row we win outright, exactly as M4A was.

And the open research question the phase exists to answer: **no published tool achieves fingerprint-resistance for PDF without destroying the document's text layer.** We intend to characterise that frontier honestly — measure where the boundary actually sits, and publish it as a trade-off, rather than assert a win.

6.2 What is already measured

The phase opened with a groundwork spike, and it has already produced results that changed the design:

- **We ran our own fingerprint guard against the industry-standard PDF library (qpdf) and it failed.** The library stamps a constant signature into every file it writes, spanning both the

file header and the object layout — so a document cleaned through it announces which library cleaned it.

- **A sharper finding that was not on our original list:** a PDF's *document identifier* is **inherited from the input and survives** a standard rewrite. A file cleaned that way still carries the identifier linking it to every other revision of the same document — a straightforward leak that a tags-only clean leaves completely intact.
- **We measured what MAT2 actually does to a PDF**, rather than repeating what is commonly said. The widely-repeated claim that "MAT2 destroys PDF text" is true only of its default path, which renders every page to an image and quadruples the file size; its lightweight path preserves text fine. So we narrowed our own benchmark claim to what we can defend: **MAT2 offers no bit-preserving and no lossless tier for PDF at all — both of its paths are re-renders.** Narrowing our own claim before publishing it is the standard this project holds itself to.

The resulting decision: we write the PDF serializer ourselves. We keep the standard library as the reader and semantic layer, but we emit the bytes. This is more work, and it is the only way to reach the same standard of claim for PDF that PNG and FLAC already meet — otherwise we would be publishing a materially weaker promise and arguing past our own guard.

6.3 Phase 3 milestones

	Deliverable	Status
M0	Groundwork spike; serializer decision taken and recorded	✅ Done
M1	PDF corpus + proof that incremental-update history is provably gone	➡ SOON Next
M2	PDF structure walker, serializer, content tokenizer; **recursive** F1 — clearing the document's own metadata *and* the metadata inside every image it embeds	➡ SOON
M3	Measure which channel actually leaks, **before** designing the fix	➡ SOON
M4	PDF F2 + the honest fingerprint answer, whatever it turns out to be	➡ SOON
M5	PDF F3 (rasterise) + a **redaction-risk detector** that warns users when text is still hiding under a black box	➡ SOON
M6	Word/DOCX walker + F1, with correct ZIP timestamp handling	➡ SOON
M7	RSIDs measurably cleared where MAT2 leaves them	➡ SOON

Note the ordering: **M3 comes before M4** deliberately. Designing the fix before measuring what leaks is normalising by guesswork, and we would then have no way to tell a real limit from unfinished work.

6.4 Limits we already expect to publish from this phase

Recorded now as predictions to be tested, not as conclusions:

- **Redaction is not scrubbing.** Text hidden under a black rectangle stays in the document. We will detect and warn; we will not silently repair, and users must be told plainly because they assume otherwise.
- **Rasterising a PDF destroys selectable text** — it can no longer be searched, copied from, or read aloud by a screen reader. A real cost to a real user, and it goes in the limits register *before* the feature is built.
- **A signed PDF cannot be cleaned and stay signed.** Any rewrite invalidates the signature. Unavoidable, and better said up front.

---

7. Far future — the road to the finished tool

The end goal is one tool that irreversibly scrubs files of **arbitrary type**. The remaining phases, in dependency order:

Phase 4 — media and camera formats (MP4 → HEIC → RAW). Video and modern phone photos share the same internal box structure we already built for M4A, so MP4 and HEIC reuse existing code. Camera RAW files are the hardest: they embed a **full-size JPEG preview and a thumbnail, each carrying its own complete set of camera and GPS data** — a major leak, and one our image handlers are already built to clean recursively.

Phase 5 — executables (Windows, Linux, macOS binaries). A separate universe with no shared code: compiler toolchain fingerprints, source-code paths left in debug records, build identifiers, and unique binary IDs. The hard constraint is that the program must still run identically afterwards.

Phase 6 — the long tail and whole-pipeline integration. SVG, TIFF, GIF, WebP, EPUB and similar, most of which reuse the standards modules already written. Then final integration: one dispatcher across every handler, batch processing, and **recursive container cleaning** — a document inside an archive inside an email attachment, cleaned all the way down.

The parallel deliverable: a professional funding proposal — technical content backed by the measured guarantee matrices, plus a cost plan. Every phase feeds it directly: the matrices *are* the evidence base, which is why they are generated by machine and re-verified on every build rather than written by hand.

The end goal, stated precisely: forensic unrecoverability, validated by differential testing at **every** known hiding place, per format, against the medium-tier adversary — with every residual that cannot be removed documented in plain words rather than quietly omitted.

---

8. How this project works, and why it is ordered this way

Three principles explain most of the decisions above, and they are worth stating because they are what makes the results trustworthy.

Leaves before containers. File formats nest: a PDF contains JPEGs, a Word document contains images and a rendered preview, a RAW photo contains two JPEGs, a video contains cover art. A container can only be cleaned properly once the formats it embeds can be cleaned. That is why images and audio came first and documents come now — not because they are more popular, but because everything else depends on them. Building PDF first would have forced us to destroy every document's text layer to claim success.

Shared code, written once. The EXIF, XMP, ICC and box-structure modules are written once and called by every format handler. A copy-pasted parser is how a missed update becomes a leak in one format and not another.

No claim without an experiment. Feasibility is expressed as a per-format matrix of what is reachable at which cost — never a binary "yes it's clean". Every cell carries the verdict *and* the reason and residuals behind it. The build re-measures all of it and fails if a published claim has drifted. When we have found our own claims to be broader than our evidence — twice so far — we have narrowed them in writing rather than leaving them standing.

---

9. Status at a glance

Phases complete	0 (foundation), 1 (images), 2 (audio)
Phase in progress	3 (documents) — groundwork measured, design decision taken
Formats shipping	JPEG, PNG, MP3, FLAC, M4A
Automated tests	243, on four Python versions, on every change
Published claims	Re-measured from scratch by machine on every build
Known limits	12, published in plain language and rendered into every report
Capabilities no surveyed competitor has	M4A support at all · lossless-and-untraceable (PNG, FLAC) · user-chosen fidelity · a measured guarantee matrix